

Nama : Navassa Anastha

NIM : 23041450080

MK : Artificial Intelijen dan Sistem Data

Kelas : 2383H

Link Github: <https://github.com/navassaanastha/prediksi-kelulusan-mahasiswa.git>

Hasil Prediksi Kelulusan Tepat Waktu

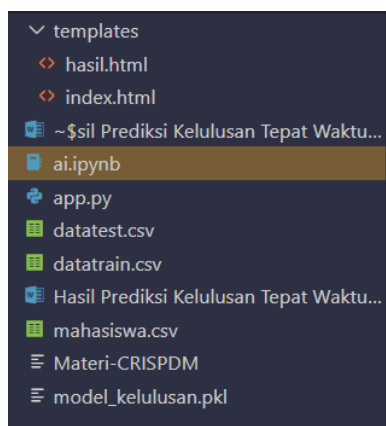
```
> <
# IMPORT LIBRARY
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv('mahasiswa.csv')

df.head()
```

(1)

	tahun_masuk	bulan_lulus	ipk	jumlah_mk_gagal	sks_lulus	status_skripsi	jumlah_bimbingan	ontime
0	2018	11	3.21	2	128	2	8	1
1	2017	6	3.62	0	111	4	12	1
2	2018	5	3.66	0	134	3	9	1
3	2018	12	3.69	1	124	3	13	1
4	2018	7	2.99	7	116	0	7	1



1. Business Understanding

Pada tahap Business Understanding dijelaskan permasalahan utama yaitu masih sulitnya pihak kampus dalam memantau mahasiswa yang berpotensi tidak lulus tepat waktu. Oleh karena itu dibuat sistem prediksi kelulusan mahasiswa menggunakan machine learning untuk membantu proses monitoring akademik dan memberikan early warning terhadap mahasiswa beresiko.

Tujuan penelitian ini adalah membangun sistem prediksi kelulusan mahasiswa tepat waktu menggunakan algoritma Decision Tree berbasis Flask.

2. Data Understanding

Pada tahap Data Understanding dilakukan pemahaman terhadap dataset mahasiswa. Dataset berisi 600 data mahasiswa dengan beberapa atribut seperti:

- 1) tahun masuk

- 2) bulan lulus
- 3) IPK
- 4) jumlah MK gagal
- 5) SKS lulus
- 6) status skripsi
- 7) jumlah bimbingan
- 8) ontime

Berdasarkan hasil eksplorasi data pada halaman Data Understanding terlihat bahwa:

- 1) dataset memiliki 600 baris dan 8 kolom
- 2) tidak terdapat missing value
- 3) tipe data sudah sesuai
- 4) target prediksi adalah kolom ontime

Keterangan target:

- 1) 1 = lulus tepat waktu
- 2) 0 = tidak lulus tepat waktu

Pada tahap ini juga dilakukan analisis distribusi data target menggunakan bar chart untuk melihat perbandingan jumlah mahasiswa ontime dan tidak ontime.

```

Data Understanding

# 1. IDENTIFIKASI STRUKTUR DATASET

print("Jumlah Baris dan Kolom Dataset:")
print(df.shape)

print("\nInformasi Dataset:")
df.info()

print("\nStatistik Dataset:")
display(df.describe())

print("\nCek Missing Value:")
print(df.isnull().sum())

```

```

Jumlah Baris dan Kolom Dataset:
(600, 8)

Informasi Dataset:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 600 entries, 0 to 599
Data columns (total 8 columns):
 #   Column             Non-Null Count  Dtype  
---  --
 0   tahun_masuk        600 non-null    int64  
 1   bulan_lulus         600 non-null    int64  
 2   ipk                 600 non-null    float64
 3   jumlah_mk_gagal     600 non-null    int64  
 4   sks_lulus           600 non-null    int64  
 5   status_skripsi      600 non-null    int64  
 6   jumlah_bimbingan    600 non-null    int64  
 7   ontime              600 non-null    int64  
dtypes: float64(1), int64(7)
memory usage: 37.6 KB

Statistik Dataset:

```

	tahun_masuk	bulan_lulus	ipk	jumlah_mk_gagal	sks_lulus	status_skripsi	jumlah_bimbingan	ontime
count	600.000000	600.000000	600.000000	600.000000	600.000000	600.000000	600.000000	600.000000
mean	2017.496667	6.398333	3.001917	3.106667	129.791667	1.890000	6.718333	0.438333
std	1.720928	3.389515	0.446323	2.171747	11.637898	1.276956	3.513247	0.496597
min	2015.000000	1.000000	2.000000	0.000000	110.000000	0.000000	1.000000	0.000000
25%	2016.000000	4.000000	2.740000	1.000000	120.000000	1.000000	4.000000	0.000000
50%	2018.000000	6.000000	3.010000	3.000000	130.000000	2.000000	7.000000	0.000000
75%	2019.000000	9.000000	3.290000	5.000000	140.000000	3.000000	9.000000	1.000000
max	2020.000000	12.000000	4.000000	7.000000	149.000000	4.000000	14.000000	1.000000

```

Cek Missing Value:
tahun_masuk      0
bulan_lulus      0
ipk              0
jumlah_mk_gagal  0
sks_lulus        0
status_skripsi   0
jumlah_bimbingan 0
ontime           0
dtype: int64

```

```
# 2. IDENTIFIKASI FEATURE (X) DAN TARGET / LABEL (Y)
# fitur (x) untuk data input prediksi
# label (y) untuk hasil klasifikasi/prediksi

# Feature / Kriteria / History
X = df[['tahun_masuk',
        'bulan_lulus',
        'ipk',
        'jumlah_mk_gagal',
        'sks_lulus',
        'status_skripsi',
        'jumlah_bimbingan']]

# Target / Label
y = df['ontime']

print("===== FEATURE (X) =====")
display(X.head())

print("\n===== TARGET / LABEL (Y) =====")
display(y.head())

print("\nKeterangan Target:")
print("Ontime = 1 --> Lulus <= 4 Tahun")
print("Ontime = 0 --> Lulus > 4 Tahun")
```

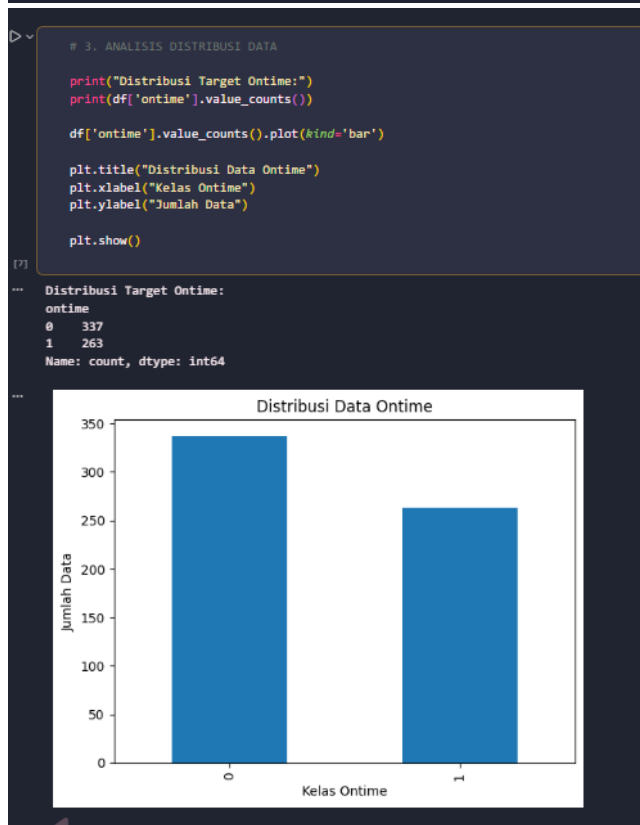
===== FEATURE (X) =====

	tahun_masuk	bulan_lulus	ipk	jumlah_mk_gagal	sks_lulus	status_skripsi	jumlah_bimbingan
0	2018	11	3.21	2	128	2	8
1	2017	6	3.62	0	111	4	12
2	2018	5	3.66	0	134	3	9
3	2018	12	3.69	1	124	3	13
4	2018	7	2.99	7	116	0	7

===== TARGET / LABEL (Y) =====

```
0    1
1    1
2    1
3    1
4    1
Name: ontime, dtype: int64
```

Keterangan Target:
Ontime = 1 --> Lulus <= 4 Tahun
Ontime = 0 --> Lulus > 4 Tahun



```
# 4. RULE-BASED SYSTEM

df['rule_based_ontime'] = (
    (df['ipk'] >= 3.00) &
    (df['jumlah_mk_gagal'] <= 2) &
    (df['jumlah_himbingan'] > 10) &
    (df['status_skripsi'] < 7)
).astype(int)

print("Hasil Rule-Based System:")
display(df[['ipk',
            'jumlah_mk_gagal',
            'jumlah_himbingan',
            'status_skripsi',
            'rule_based_ontime']].head(10))

print("\nDistribusi Rule-Based:")
print(df['rule_based_ontime'].value_counts())

print("\nKeterangan:")
print("Ontime = 1 jika memenuhi rule")
print("Ontime = 0 jika tidak memenuhi rule")

# mengambil hanya data yang memenuhi rule (nilai = 1)
data_rule_1 = df[df['rule_based_ontime'] == 1]
display(data_rule_1.head(10))
```

Hasil Rule-Based System:

	ipk	jumlah_mk_gagal	jumlah_himbingan	status_skripsi	rule_based_ontime
0	3.21	2	8	2	0
1	3.62	0	12	4	1
2	3.66	0	9	3	0
3	3.69	1	13	3	1
4	2.99	7	7	0	0
5	3.16	2	2	0	0
6	3.51	0	8	3	0
7	3.17	2	2	0	0
8	3.10	3	9	1	0
9	3.40	0	10	3	0

Distribusi Rule-Based:

```
rule_based_ontime
0    532
1     68
Name: count, dtype: int64
```

Keterangan:

```
Ontime = 1 jika memenuhi rule
Ontime = 0 jika tidak memenuhi rule
```

	tahun_masuk	bulan_hulus	ipk	jumlah_mk_gagal	skrs_hulus	status_skripsi	jumlah_himbingan	ontime	rule_based_ontime
1	2017	6	3.62	0	111	4	12	1	1
3	2018	12	3.69	1	124	3	13	1	1
17	2017	3	3.16	2	137	3	11	1	1
24	2017	8	3.36	1	114	4	12	1	1
28	2020	10	3.37	1	111	4	12	0	1
30	2020	8	3.38	1	134	4	11	1	1
34	2015	12	3.38	1	128	3	14	1	1
44	2019	12	3.25	2	141	3	13	1	1
47	2019	5	3.75	0	125	3	14	1	1

3. Data Preparation

Pada tahap Data Preparation dilakukan proses pembersihan dan persiapan data sebelum digunakan pada machine learning. Tahapan yang dilakukan yaitu:

- 1) pengecekan missing value
- 2) pengecekan data duplikat
- 3) konversi tipe data
- 4) feature engineering
- 5) split data training dan testing

Awalnya dilakukan normalisasi menggunakan MinMaxScaler, namun pada implementasi deployment model akhir normalisasi tidak digunakan karena Decision Tree tidak sensitif terhadap skala data dan agar input dashboard Flask sesuai dengan data asli mahasiswa.

Selain itu dilakukan feature engineering untuk menambahkan fitur baru seperti:

- 1) rata_ip
- 2) progress_skripsi_sks
- 3) tren_ip
- 4) Dataset kemudian dibagi menjadi:
- 5) 80% data training
- 6) 20% data testing

Hasil split data disimpan ke:

- 1) datatrain.csv
- 2) datatest.csv

Data preparation

```
# 1. DATA CLEANING

# cek missing value / data kosong
print("Cek Missing Value:")
print(df.isnull().sum())

# hapus missing value
df = df.dropna()

# cek data duplikat
print("\nJumlah Data Duplikat:")
print(df.duplicated().sum())

# hapus data duplikat
df = df.drop_duplicates()

# konversi format data
df['tahun_masuk'] = df['tahun_masuk'].astype(int)
df['bulan_lulus'] = df['bulan_lulus'].astype(int)
df['jumlah_mk_gagal'] = df['jumlah_mk_gagal'].astype(int)
df['sks_lulus'] = df['sks_lulus'].astype(int)
df['status_skripsi'] = df['status_skripsi'].astype(int)
df['jumlah_bimbingan'] = df['jumlah_bimbingan'].astype(int)
df['ontime'] = df['ontime'].astype(int)

# tampilkan hasil setelah cleaning
print("\nJumlah Data Setelah Cleaning:")
print(df.shape)

print("\nTipe Data Setelah Cleaning:")
print(df.dtypes)

print("\nData Setelah Cleaning:")
display(df.head())
```

Cek Missing Value:

```
tahun_masuk      0
bulan_lulus      0
ipk              0
jumlah_mk_gagal  0
sks_lulus        0
status_skripsi   0
jumlah_bimbingan 0
ontime           0
rule_based_ontime 0
dtype: int64
```

Jumlah Data Duplikat:

0

Jumlah Data Setelah Cleaning:

(600, 9)

Tipe Data Setelah Cleaning:

```
tahun_masuk      int64
bulan_lulus      int64
ipk              float64
jumlah_mk_gagal  int64
sks_lulus        int64
status_skripsi   int64
...
rule_based_ontime int64
dtype: object
```

Data Setelah Cleaning:

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#).

	tahun_masuk	bulan_lulus	ipk	jumlah_mk_gagal	sks_lulus	status_skripsi	jumlah_bimbingan	ontime	rule_based_ontime
0	2018	11	3.21	2	128	2	8	1	0
1	2017	6	3.62	0	111	4	12	1	1
2	2018	5	3.66	0	134	3	9	1	0
3	2018	12	3.69	1	124	3	13	1	1
4	2018	7	2.99	7	116	0	7	1	0

Generate

```
# 2. Data Transformation

# ENCODING DATA KATEGORIKAL
# mengganti status_skripsi bertipe integer
df['status_skripsi'] = df['status_skripsi'].astype(int)

print("Tipe Data Setelah Encoding:")
print(df.dtypes)

# NORMALISASI DATA
from sklearn.preprocessing import MinMaxScaler
# membuat object normalisasi
scaler = MinMaxScaler()

# kolom yang dinormalisasi
kolom_normalisasi = [
    'tahun_masuk',
    'bulan_lulus',
    'ipk',
    'jumlah_mk_gagal',
    'sks_lulus',
    'status_skripsi',
    'jumlah_bimbingan',
]

# proses normalisasi
df[kolom_normalisasi] = scaler.fit_transform(df[kolom_normalisasi])

print("Data Setelah Normalisasi:")
display(df.head())
```

Tipe Data Setelah Encoding:

```
tahun_masuk      int64
bulan_lulus      int64
ipk              float64
jumlah_mk_gagal  int64
sks_lulus        int64
status_skripsi   int64
jumlah_bimbingan int64
ontime           int64
rule_based_ontime int64
dtype: object
```

Data Setelah Normalisasi:

	tahun_masuk	bulan_lulus	ipk	jumlah_mk_gagal	sks_lulus	status_skripsi	jumlah_bimbingan	ontime	rule_based_ontime
0	0.6	0.909091	0.605	0.285714	0.461538	0.50	0.538462	1	0
1	0.4	0.454545	0.810	0.000000	0.025641	1.00	0.846154	1	1
2	0.6	0.363636	0.830	0.000000	0.615385	0.75	0.615385	1	0
3	0.6	1.000000	0.845	0.142857	0.358974	0.75	0.923077	1	1
4	0.6	0.545455	0.495	1.000000	0.153846	0.00	0.461538	1	0

```
# 3. FEATURE ENGINEERING

# membuat fitur rata IP
df['rata_ip'] = df['ipk']

# membuat fitur progress skripsi / SKS
df['progress_skripsi_sks'] = (
    df['status_skripsi'] + df['sks_lulus']
) / 2

# membuat fitur tren IP
df['tren_ip'] = df['ipk'] * 1.05

print("Fitur Baru Hasil Feature Engineering:")
display(df[['ipk',
             'sks_lulus',
             'status_skripsi',
             'rata_ip',
             'progress_skripsi_sks',
             'tren_ip']].head())
```

Fitur Baru Hasil Feature Engineering:

	ipk	sks_lulus	status_skripsi	rata_ip	progress_skripsi_sks	tren_ip
0	0.605	0.461538	0.50	0.605	0.480769	0.63525
1	0.810	0.025641	1.00	0.810	0.512821	0.85050
2	0.830	0.615385	0.75	0.830	0.682692	0.87150
3	0.845	0.358974	0.75	0.845	0.554487	0.88725
4	0.495	0.153846	0.00	0.495	0.076923	0.51975

```
# 4. FEATURE SELECTION

# memilih fitur yang relevan
fitur_relevan = [
    'tahun_masuk',
    'bulan_lulus',
    'ipk',
    'jumlah_mk_gagal',
    'sks_lulus',
    'status_skripsi',
    'jumlah_bimbingan',
    'rata_ip',
    'progress_skripsi_sks',
    'tren_ip'
]

# feature / input
x = df[fitur_relevan]

# target / label
y = df['ontime']

print("Feature yang digunakan:")
display(x.head())

print("\nTarget / Label:")
display(y.head())
```

Feature yang digunakan:

	tahun_masuk	bulan_lulus	ipk	jumlah_mk_gagal	sks_lulus	status_skripsi	jumlah_bimbingan	rata_ip	progress_skripsi_sks	tren_ip
0	0.6	0.909091	0.605	0.285714	0.461538	0.50	0.538462	0.605	0.480769	0.63525
1	0.4	0.454545	0.810	0.000000	0.025641	1.00	0.846154	0.810	0.512821	0.85050
2	0.6	0.363636	0.830	0.000000	0.615385	0.75	0.615385	0.830	0.682692	0.87150
3	0.6	1.000000	0.845	0.142857	0.358974	0.75	0.923077	0.845	0.554487	0.88725
4	0.6	0.545455	0.495	1.000000	0.153846	0.00	0.461538	0.495	0.076923	0.51975

Target / Label:

	ontime
0	1
1	1
2	1
3	1
4	1

Name: ontime, dtype: int64

4. Modeling

Pada tahap Modeling digunakan algoritma:

Decision Tree Classifier

Algoritma ini dipilih karena:

- 1) mudah dipahami
- 2) cocok untuk klasifikasi
- 3) mampu membaca pola akademik mahasiswa
- 4) hasil dapat divisualisasikan dalam bentuk pohon keputusan

Model dilatih menggunakan parameter:

DecisionTreeClassifier(max_depth=5, random_state=42)

Model kemudian dilatih menggunakan data training:

```
model.fit(X_train, y_train)
```

Setelah model berhasil dilatih, model disimpan menggunakan:

```
joblib.dump()
```

menjadi file:

```
model_kelulusan.pkl
```

File tersebut digunakan kembali pada tahap deployment Flask tanpa perlu training ulang.

```
data.csv
datatrain.csv

Modeling (Machine Learning)

# 1. Split Data
from sklearn.model_selection import train_test_split

# feature dan target
X = df[['tahun_masuk',
        'bulan_lulus',
        'ipk',
        'jumlah_mk_gagal',
        'sks_lulus',
        'status_skripsi',
        'jumlah_hibingman',
        'rata_ip',
        'progress_skripsi_sks',
        'tren_ip']]

y = df['ontime']

# split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42)

# gabung feature + target
train_data = X_train.copy()
train_data['ontime'] = y_train

test_data = X_test.copy()
test_data['ontime'] = y_test

# simpan csv
train_data.to_csv('datatrain.csv', index=False) #latih model
test_data.to_csv('datatest.csv', index=False) #menguji model

# output
print("Training:", train_data.shape)
print("Testing :", test_data.shape)

display(train_data.head())
display(test_data.head())

Training: (480, 11)
Testing : (120, 11)

   tahun_masuk  bulan_lulus  ipk  jumlah_mk_gagal  sks_lulus  status_skripsi  jumlah_hibingman  rata_ip  progress_skripsi_sks  tren_ip  ontime
145          1.0    0.909091  0.660      0.000000  0.848154          1.00      0.538462      0.660      0.923077  0.69300      1
9            0.8    0.090909  0.700      0.000000  0.589744          0.75      0.692308      0.700      0.669872  0.73500      1
375          1.0    0.636364  0.330      0.714286  0.717940          0.25      0.461538      0.330      0.483974  0.34650      0
523          0.0    0.636364  0.585      0.285714  0.666667          0.25      0.615385      0.585      0.458333  0.61425      1
188          0.2    0.454545  0.905      0.142857  0.358974          0.75      0.923077      0.905      0.554487  0.95025      1

   tahun_masuk  bulan_lulus  ipk  jumlah_mk_gagal  sks_lulus  status_skripsi  jumlah_hibingman  rata_ip  progress_skripsi_sks  tren_ip  ontime
110          0.4    0.636364  0.300      1.000000  0.820513          0.25      0.461538      0.300      0.535256  0.31500      0
419          0.6    0.545455  0.630      0.428571  0.153548          1.00      0.769231      0.630      0.576923  0.66150      1
565          0.4    0.818182  0.455      0.857143  1.000000          0.25      0.000000      0.455      0.655000  0.47775      0
77           0.8    0.818182  0.830      0.142857  0.661538          0.75      0.661538      0.830      0.657500  0.37650      1
181          0.4    0.090909  0.495      0.857143  0.153548          0.50      0.307692      0.495      0.326923  0.51975      0
```

```
# 2. Training Model / Model AI
from sklearn.tree import DecisionTreeClassifier

# membuat model Decision Tree
model = DecisionTreeClassifier(
    max_depth=5,
    random_state=42
)

# melatih model menggunakan data training
model.fit(X_train, y_train)

print("Model Decision Tree berhasil dilatih")

Model Decision Tree berhasil dilatih

# 3. PREDIKSI DATA TESTING

# melakukan prediksi
y_pred = model.predict(X_test)

print("Hasil Prediksi:")
print(y_pred[:10])

[15]
*** Hasil Prediksi:
[0 1 0 1 0 1 0 1 0 1]
```

5. Evaluation

Pada tahap Evaluation dilakukan pengujian performa model menggunakan:

- 1) Accuracy
- 2) Precision
- 3) Recall

4) Confusion Matrix

Hasil evaluasi model:

- 1) Accuracy = 70.83%
- 2) Precision = 67.92%
- 3) Recall = 66.67%

Berdasarkan hasil tersebut model sudah cukup baik dalam melakukan prediksi kelulusan mahasiswa tepat waktu.

Confusion Matrix menunjukkan:

- 1) 49 data berhasil diprediksi tidak ontime dengan benar
- 2) 36 data berhasil diprediksi ontime dengan benar
- 3) masih terdapat beberapa kesalahan prediksi namun model masih cukup stabil dan tidak mengalami overfitting berat.

Selain itu dilakukan visualisasi accuracy menggunakan bar chart untuk melihat performa model secara visual.

```
# 1. metode evaluasi

# ACCURACY
from sklearn.metrics import accuracy_score
# menghitung accuracy
accuracy = accuracy_score(y_test, y_pred)
# ubah ke persen
accuracy_persen = accuracy * 100
print("Nilai Accuracy:")
print(f"{accuracy_persen:.2f}%")

# PRECISION
from sklearn.metrics import precision_score
# menghitung precision
precision = precision_score(y_test, y_pred)
precision_persen = precision * 100
print("Nilai Precision:")
print(f"Precision: {precision_persen:.2f}%")

# RECALL
from sklearn.metrics import recall_score
# menghitung recall
recall = recall_score(y_test, y_pred)
recall_persen = recall * 100
print("Nilai Recall:")
print(f"Recall : {recall_persen:.2f}%")

# CONFUSION MATRIX
from sklearn.metrics import confusion_matrix
# membuat confusion matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)
```

[20]

```
... Nilai Accuracy:
70.83%
Nilai Precision:
Precision: 67.92%
Nilai Recall:
Recall : 66.67%
Confusion Matrix:
[[49 17]
 [18 36]]
```



6. Insight / Informasi

Pada tahap Insight dilakukan analisis feature importance untuk mengetahui faktor yang paling mempengaruhi kelulusan mahasiswa.

Berdasarkan hasil feature importance diperoleh:

- 1) IPK menjadi faktor paling berpengaruh
- 2) jumlah bimbingan memiliki pengaruh cukup besar
- 3) SKS lulus dan progress skripsi juga mempengaruhi kelulusan mahasiswa

Pola mahasiswa beresiko:

- 1) IPK rendah
- 2) jumlah MK gagal tinggi
- 3) jumlah bimbingan sedikit
- 4) progres skripsi lambat

Hasil ini menunjukkan bahwa performa akademik dan progres skripsi sangat mempengaruhi kemungkinan mahasiswa lulus tepat waktu.



7. Deployment

Pada tahap Deployment dibuat sistem prediksi berbasis web menggunakan:
Framework Flask

Sistem memiliki fitur:

- 1) dashboard prediksi mahasiswa
- 2) hasil prediksi kelulusan
- 3) rekomendasi tindakan
- 4) API JSON AI

User dapat menginput data mahasiswa melalui dashboard web, kemudian sistem akan memproses prediksi menggunakan model `model_kelulusan.pkl`.

Hasil prediksi:

- 1) mahasiswa lulus tepat waktu
- 2) mahasiswa beresiko tidak lulus tepat waktu
- 3) Sistem juga memberikan rekomendasi tindakan akademik berdasarkan hasil prediksi.

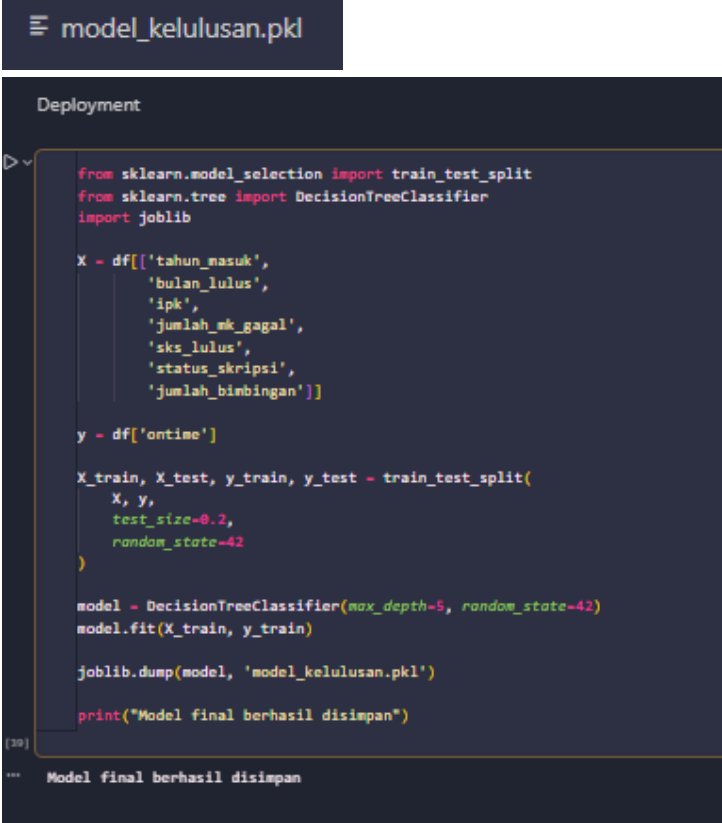
Selain dashboard web, sistem juga memiliki:

JSON API

yang dapat diakses melalui:

127.0.0.1:5000/api/predict

API ini digunakan untuk menampilkan hasil prediksi machine learning dalam format JSON sehingga model AI dapat diakses oleh sistem atau aplikasi lain.



```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
import joblib

X = df[['tahun_masuk',
        'bulan_lulus',
        'ipk',
        'jumlah_mk_gagal',
        'sks_lulus',
        'status_skripsi',
        'jumlah_bimbingan']]

y = df['ontime']

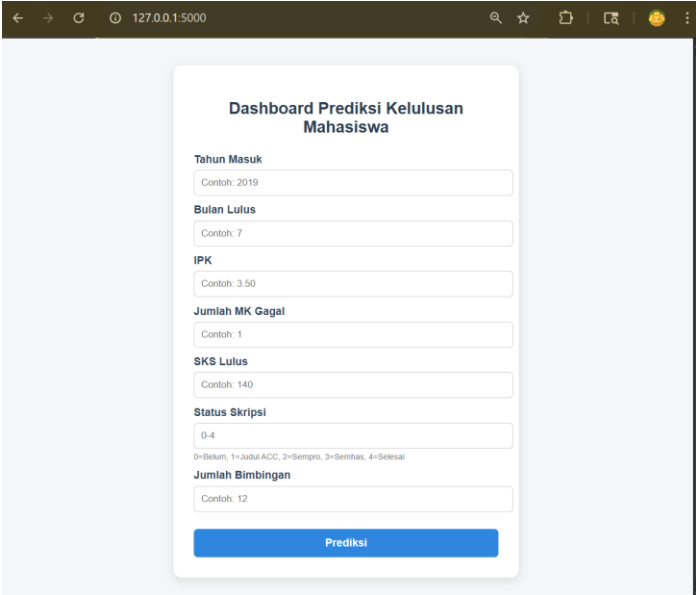
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

model = DecisionTreeClassifier(max_depth=5, random_state=42)
model.fit(X_train, y_train)

joblib.dump(model, 'model_kelulusan.pkl')

print("Model final berhasil disimpan")
```

Model final berhasil disimpan



Dashboard Prediksi Kelulusan Mahasiswa

Tahun Masuk
Contoh: 2019

Bulan Lulus
Contoh: 7

IPK
Contoh: 3.50

Jumlah MK Gagal
Contoh: 1

SKS Lulus
Contoh: 140

Status Skripsi
0-4
0=Belum, 1=Judul ACC, 2=Sempurna, 3=Semtias, 4=Selesai

Jumlah Bimbingan
Contoh: 12

Prediksi

```
127.0.0.1:5000/api/predict

{
  "hasil_prediksi": "LULUS TEPAT WAKTU",
  "input_data": {
    "bulan_lulus": 7,
    "ipk": 3.8,
    "jumlah_bimbingan": 14,
    "jumlah_mk_gagal": 0,
    "sks_lulus": 145,
    "status_skripsi": 4,
    "tahun_masuk": 2019
  },
  "nilai_prediksi": 1,
  "rekomendasi": "Pertahankan performa akademik dan progres skripsi."
}
```

Dashboard Prediksi Kelulusan Mahasiswa

Tahun Masuk

2018

Bulan Lulus

11

IPK

3.21

Jumlah MK Gagal

2

SKS Lulus

128

Status Skripsi

2

0=Belum, 1=Judul ACC, 2=Sempro, 3=Semhas, 4=Selesai

Jumlah Bimbingan

8

Prediksi

Hasil Prediksi Kelulusan

Mahasiswa diprediksi LULUS TEPAT WAKTU

Pertahankan performa akademik dan progres skripsi.

Kembali

Dashboard Prediksi Kelulusan Mahasiswa

Tahun Masuk
2015

Bulan Lulus
12

IPK
3.16

Jumlah MK Gagal
2

SKS Lulus
118

Status Skripsi
0
0=Belum, 1=Judul ACC, 2=Sempurna, 3=Seminar, 4=Selesai

Jumlah Bimbingan
2

Prediksi

Hasil Prediksi Kelulusan

Mahasiswa BERESIKO TIDAK LULUS TEPAT WAKTU

Perlu monitoring akademik, peningkatan bimbingan, dan evaluasi mata kuliah gagal.

Kembali

8. Simpulan

Pada tugas ini berhasil dibuat sistem prediksi kelulusan mahasiswa tepat waktu menggunakan algoritma Decision Tree dengan tahapan CRISP-DM. Dataset mahasiswa digunakan untuk melatih model machine learning berdasarkan beberapa atribut akademik seperti IPK, jumlah MK gagal, SKS lulus, status skripsi, dan jumlah bimbingan.

Hasil evaluasi model menunjukkan accuracy sebesar 70.83%, precision sebesar 67.92%, dan recall sebesar 66.67%, sehingga model cukup baik dalam melakukan prediksi mahasiswa tepat waktu dan tidak tepat waktu.

Selain itu, berhasil dibuat deployment berbasis web menggunakan framework Flask yang memiliki fitur dashboard prediksi, hasil prediksi, rekomendasi tindakan, dan JSON API. Berdasarkan hasil feature importance diketahui bahwa IPK, jumlah bimbingan, dan SKS lulus menjadi faktor yang paling berpengaruh terhadap kelulusan mahasiswa tepat waktu.

Link Github:

<https://github.com/navassaanastha/prediksi-kelulusan-mahasiswa>